

Karachi AQI Predictor

End-to-End Serverless Machine Learning Pipeline

Intern
Rashid Hussain

Programme
10Pearls Shine Internship

Live Dashboard:
<https://aqipredictorbyrashidhussain.streamlit.app/>

Repository:
<https://github.com/rh29152/AQI-Predictor>
EDA Notebook: [notebooks/01_eda.ipynb](#)

June 7, 2026

Contents

1	Introduction	3
1.1	Problem Statement	3
1.2	Why AQI Forecasting Matters	3
1.3	Project Goal: Three-Day AQI Forecasting	3
2	System Architecture	3
2.1	Technology Stack	4
3	Data Collection	4
3.1	Dataset Summary	5
3.2	Location	5
3.3	Historical Bootstrap	5
3.4	Live Updates	5
3.5	Pollutants Collected ($\mu\text{g}/\text{m}^3$ unless noted)	6
3.6	Weather Features	6
4	AQI Calculation Methodology	6
4.1	Design Choice: Pollutant-First Forecasting	6
4.2	Pipeline	7
4.3	Breakpoint Formula	7
4.4	Dominant Pollutant Convention	7
5	Feature Engineering	7
5.1	Time Features	8
5.2	Lag Features	8
5.3	Rolling Features	8
5.4	Change Rate	8
5.5	Supervised Targets (Batch Rows Only)	8
6	Exploratory Data Analysis	9
6.1	Data Quality Summary	9
6.2	Graph 1: PM2.5 & PM10 Trend Over Time	9
6.3	Graph 2: Hourly Pollution Pattern (Diurnal Cycle)	11
6.4	Graph 3: Correlation Heatmap	12
6.5	Graph 4: Pollutant / EPA AQI Distribution	12
6.6	Graph 5: SHAP Feature Importance (Dashboard)	13
6.7	EDA Summary of Insights	14
7	Model Training	14
7.1	Forecast Targets	14
7.2	Algorithms Evaluated (per Target)	14
7.3	Training Protocol	15
8	Model Evaluation	15
8.1	Best Models: Summary Table	16
8.2	Evaluation Highlights	16

9	Model Registry	16
9.1	MongoDB Metadata (per Model)	17
9.2	Hugging Face Hub	17
10	Automation / CI-CD	18
10.1	External Scheduler (cron-job.org)	19
10.2	Feature Pipeline Flow	19
10.3	Training Pipeline Flow	19
11	Dashboard	20
11.1	Example System Output	20
11.2	Dashboard Sections	21
11.3	Screenshots	21
12	Explainable AI (SHAP)	22
12.1	Method	22
12.2	Interpretation	22
13	Challenges and Solutions	23
14	Conclusion	23
14.1	Future Improvements	24
A	Repository Structure	25
B	Secrets Reference	25
	References	26

1 Introduction

1.1 Problem Statement

Karachi is one of the largest metropolitan areas in Pakistan and faces recurring air-quality stress from traffic emissions, industrial activity, construction dust, and seasonal weather patterns. Poor air quality is linked to respiratory illness, cardiovascular risk, and reduced quality of life. Citizens and planners need timely visibility into current conditions and short-range outlooks — not only historical averages.

Raw air-quality APIs often expose coarse category scales (for example OpenWeather’s 1–5 index) that do not translate directly into the EPA-style 0–500 AQI scale used in many forecasting and public-health contexts. Building a reproducible system that ingests live data, engineers temporal features, trains pollutant forecasting models, and derives AQI through breakpoint calculation for a public dashboard addresses a practical gap between data availability and actionable insight.

1.2 Why AQI Forecasting Matters

Air Quality Index (AQI) compresses multi-pollutant information into a single interpretable score. Forecasting AQI several days ahead supports:

- **Public awareness:** individuals can plan outdoor activity, ventilation, and mask use.
- **Policy and operations:** agencies can anticipate pollution episodes tied to weather or seasonality.
- **ML engineering practice:** AQI combines time-series forecasting, feature stores, model registries, and MLOps automation in one end-to-end pipeline.

1.3 Project Goal: Three-Day AQI Forecasting

This project delivers an automated **pollutant-first** forecasting system for Karachi. Rather than direct AQI prediction, the system forecasts **PM2.5**, **PM10**, **O₃**, and **NO₂** at each horizon, converts each to an AQI sub-index, and reports the **maximum sub-index** (dominant pollutant) as the final AQI.

Table 1: Forecast horizons and outputs

Horizon	Output
+24 hours	Predicted pollutant concentrations → EPA AQI
+48 hours	Predicted pollutant concentrations → EPA AQI
+72 hours	Predicted pollutant concentrations → EPA AQI

2 System Architecture

The pipeline is fully serverless: OpenWeather and Open-Meteo supply data; MongoDB Atlas stores features; GitHub Actions orchestrates ingestion and training; Hugging Face Hub stores model artefacts; Streamlit serves the dashboard.

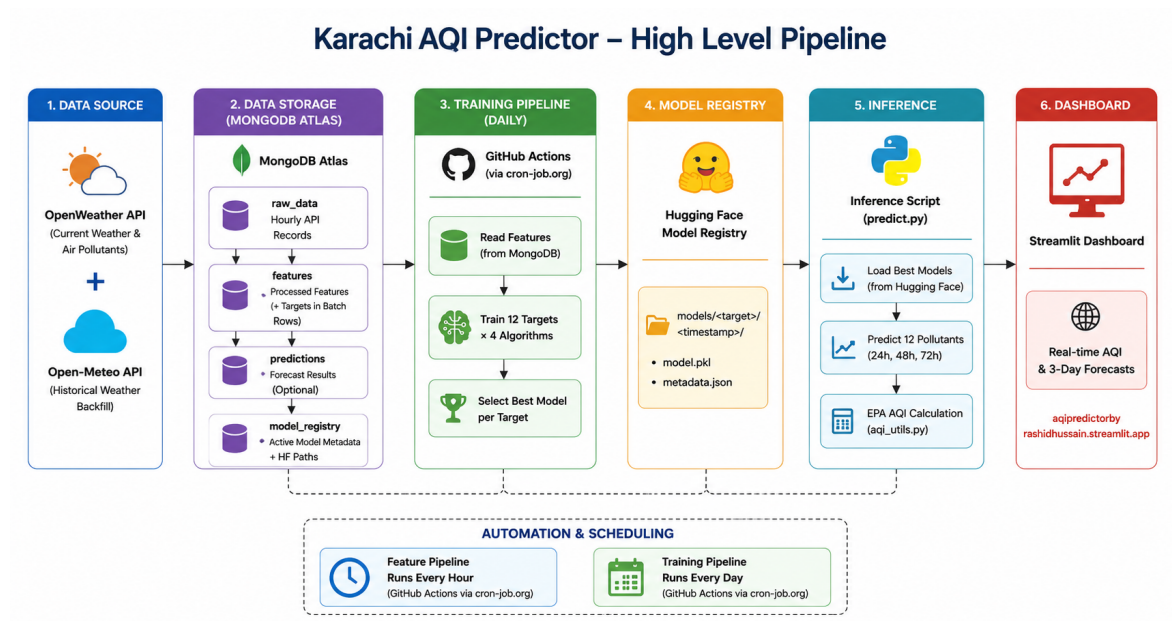


Figure 1

2.1 Technology Stack

Table 2: Technology stack

Layer	Tool
Language	Python 3.11
Data sources	OpenWeather Air Pollution API; Open-Meteo archive
Feature store	MongoDB Atlas
ML	scikit-learn (Ridge, Random Forest, Gradient Boosting), XGBoost
Model registry	Hugging Face Hub + MongoDB metadata
Explainability	SHAP
Dashboard	Streamlit, Plotly
Automation	GitHub Actions
Serialisation	joblib

3 Data Collection

3.1 Dataset Summary

Table 3: Dataset summary statistics

Metric	Value
Raw MongoDB records	2,091
Feature rows	2,089
Training-ready rows	1,971
City	Karachi
Frequency	Hourly
Historical window	Approximately 90 days
Prediction targets	12
Forecast horizons	24 h, 48 h, 72 h
Pollutants forecasted	PM2.5, PM10, O ₃ , NO ₂

Training-ready rows contain supervised target values generated during batch feature engineering. During training, each model uses rows where its specific target is available.

3.2 Location

All observations target **Karachi, Pakistan**:

Table 4: Location parameters

Parameter	Value
Latitude	24.8607° N
Longitude	67.0011° E
City label	Karachi

3.3 Historical Bootstrap

On first setup, `backfill.py` retrieves approximately **90 days** of hourly history:

1. OpenWeather air-pollution history (weekly API chunks).
2. Open-Meteo archive for matching weather fields (free, no API key).
3. Merge and upsert into MongoDB `raw_data`.
4. Batch feature engineering with supervised targets into `features`.

At the time of EDA, the store contained **2,091 raw hourly records** with **zero missing values** across pollutants and weather columns.

3.4 Live Updates

The **Feature Pipeline** runs **hourly** after every 2 hours via GitHub Actions, triggered reliably by `cron-job.org` (external scheduler). Each run:

- Optionally **catches up** missing hours (up to 48 h) using historical API fetch + incremental features.

- Fetches the **current** OpenWeather pollution + weather snapshot.
- Engineers **one incremental feature row** and upserts it to MongoDB.

Catch-up logic (up to 48 h) prevents permanent gaps when a run is delayed.

3.5 Pollutants Collected ($\mu\text{g}/\text{m}^3$ unless noted)

Table 5: Pollutant fields collected from OpenWeather

Field	Description
pm2_5	Fine particulate matter
pm10	Coarse particulate matter
o3	Ozone
no2	Nitrogen dioxide
co	Carbon monoxide
so2	Sulphur dioxide
nh3	Ammonia
aqi	OpenWeather category (1–5), stored as input signal aqi_category

3.6 Weather Features

Table 6: Weather fields collected

Field	Unit
temperature	°C
humidity	%
pressure	hPa
wind_speed	m/s
clouds	% cover

4 AQI Calculation Methodology

4.1 Design Choice: Pollutant-First Forecasting

OpenWeather exposes a **1–5 AQI category**, not a continuous EPA 0–500 score. Predicting that category directly would:

- Lose granularity across the Moderate/Poor bands.
- Ignore which pollutant drives health risk.
- Fail to align with standard breakpoint tables used in EPA-style reporting.

Solution: forecast **pollutant concentrations** first, then derive AQI mathematically.

4.2 Pipeline

AQI Derivation Pipeline

```

ML prediction (per pollutant, per horizon)
  PM2.5, PM10, O3, NO2 @ +24h / +48h / +72h
    |
    v
  EPA breakpoint interpolation (aqi_utils.py)
  sub_index = f(concentration, pollutant)
    |
    v
  Final AQI = max(sub_indices)
  dominant_pollutant = argmax(sub_indices)

```

4.3 Breakpoint Formula

For concentration C falling in breakpoint $[C_{\text{low}}, C_{\text{high}}]$ with AQI range $[I_{\text{low}}, I_{\text{high}}]$, the sub-index is computed as:

$$\text{AQI} = \frac{I_{\text{high}} - I_{\text{low}}}{C_{\text{high}} - C_{\text{low}}} \times (C - C_{\text{low}}) + I_{\text{low}} \quad (1)$$

where:

- C is the measured (or predicted) pollutant concentration,
- $C_{\text{low}}, C_{\text{high}}$ are the breakpoint concentration bounds,
- $I_{\text{low}}, I_{\text{high}}$ are the corresponding AQI bounds.

Implemented in `src/aqi_utils.py` for PM2.5, PM10, O₃, and NO₂. Negative predicted concentrations are clipped to zero before sub-index calculation.

4.4 Dominant Pollutant Convention

The reported AQI equals the **highest sub-index** among available pollutants — the same dominant-pollutant convention used in EPA-style indexing. EDA showed PM2.5 drives computed AQI on **~73%** of hours and PM10 on **~15%**, validating particulate-first modelling.

5 Feature Engineering

Features are built in two modes (`src/feature_engineering.py`):

Table 7: Feature engineering modes

Mode	When used	Targets
Batch	<code>backfill.py</code> / rebuild	12 supervised target columns included
Incremental	<code>hourly_pipeline.py</code>	Targets omitted (future values unknown)

5.1 Time Features

Table 8: Time-based features

Feature	Role
hour	Diurnal pattern (morning peaks)
day, month	Seasonal drift (spring dust)
weekday	Weak weekly effect (included for completeness)
is_weekend	Binary weekend flag

5.2 Lag Features

Per forecast pollutant: PM2.5, PM10, O₃, NO₂.

Table 9: Lag features per pollutant

Feature	Meaning
{pollutant}_lag_1	Concentration 1 hour ago
{pollutant}_lag_24	Concentration 24 hours ago
{pollutant}_lag_48	Concentration 48 hours ago

ACF analysis showed strong lag-1 persistence and visible **24 h / 48 h cycles**, justifying these lags.

5.3 Rolling Features

Rolling windows use `shift(1)` before aggregation to avoid same-hour leakage.

Table 10: Rolling window features per pollutant

Feature	Window
{pollutant}_rolling_6_mean	6 h
{pollutant}_rolling_12_mean	12 h
{pollutant}_rolling_24_mean	24 h

5.4 Change Rate

$$\text{{pollutant_change_rate}} = \frac{\text{current} - \text{lag}_{24}}{\text{lag}_{24}} \quad (2)$$

Captures rapid degradation events. EDA revealed a right-skewed PM2.5 change rate with spikes above +150%.

5.5 Supervised Targets (Batch Rows Only)

Twelve columns: `target_{pollutant}_{24|48|72}h` for pm2_5, pm10, o3, no2.

6 Exploratory Data Analysis

The complete exploratory analysis with all visualizations is available in `notebooks/01_eda.ipynb`. This section presents only key findings and insights - not every notebook figure.

6.1 Data Quality Summary

- **2,091** raw hourly records, **2,089** feature rows, **0%** missing numeric fields — no imputation required.
- Observation window: approximately **March–June 2026**.
- Fair and Moderate OpenWeather categories each **~35%** of hours; Good (1) is rare.

6.2 Graph 1: PM2.5 & PM10 Trend Over Time

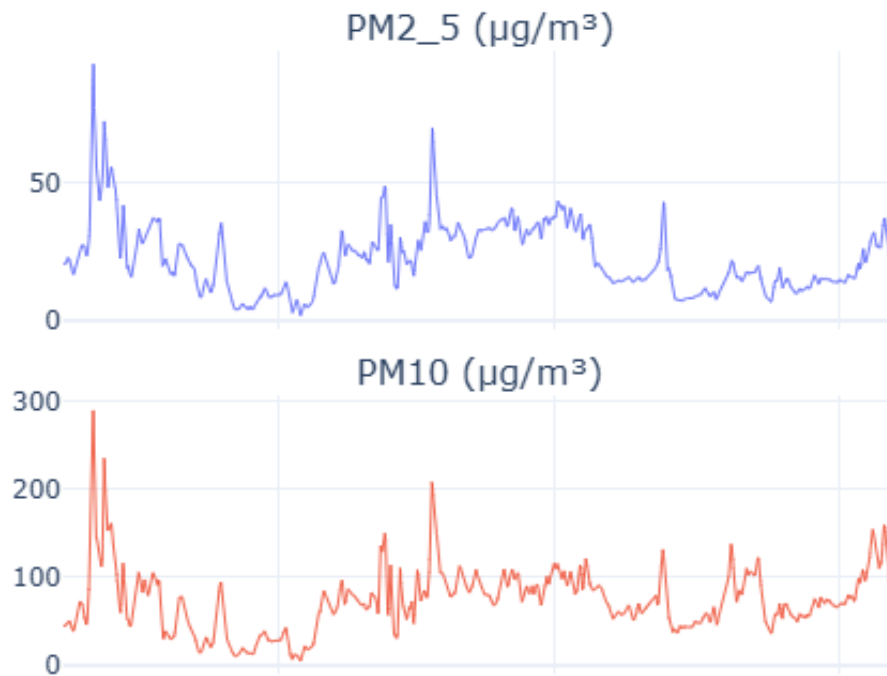


Figure 2: PM2.5 PM10 Trend Over Time

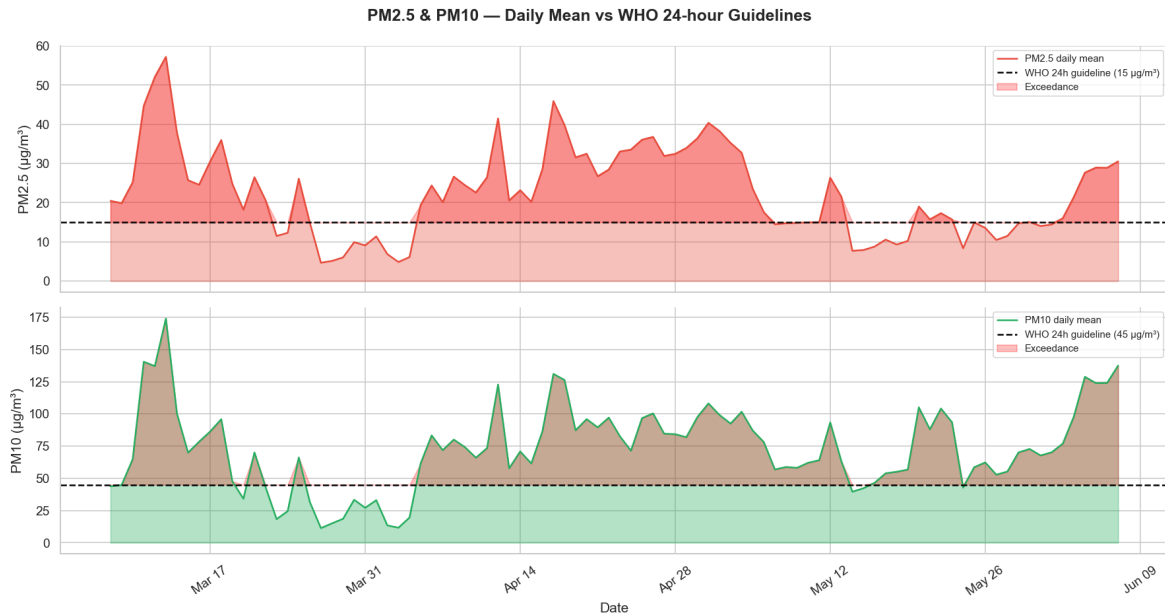


Figure 3: PM2.5 PM10 with WHO Guidelines

Finding: PM2.5 and PM10 show the largest and most abrupt spikes, especially mid-March and mid-April, while gaseous pollutants fluctuate within narrower bands. Particulates move together during spike events, suggesting shared emission sources (traffic, biomass burning, spring dust).

6.3 Graph 2: Hourly Pollution Pattern (Diurnal Cycle)

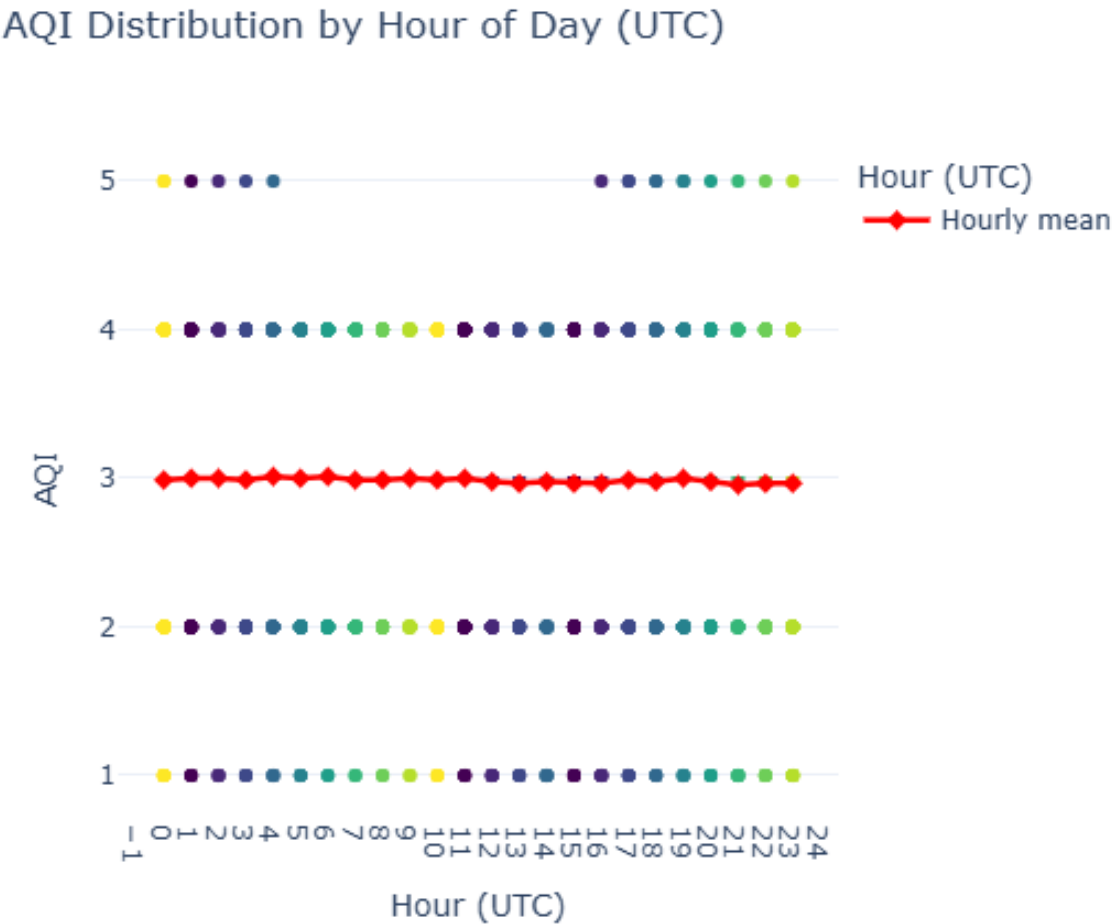


Figure 4: Hourly Pollution Pattern (Diurnal Cycle)

Finding: AQI peaks in early morning hours (~03–10 UTC), when the nocturnal boundary layer traps emissions. Conditions improve around midday (~11–14 UTC) with stronger mixing and possible sea-breeze effects. This validates `hour` as a core model input.

6.4 Graph 3: Correlation Heatmap

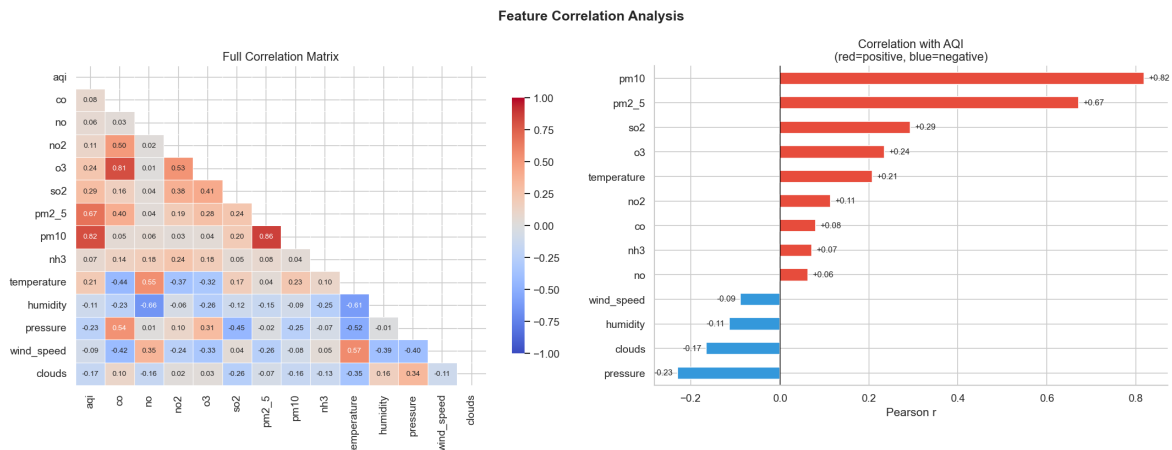


Figure 5: Correlation Heatmap

Finding:

Table 11: Key pairwise Pearson correlations

Relationship	Pearson r (approx.)
PM10 vs OpenWeather AQI	0.82
PM2.5 vs OpenWeather AQI	0.67
PM10 vs PM2.5	0.86
Wind speed vs AQI	-0.09 (weak, but Very Poor episodes at low wind)
Temperature vs AQI	+0.21
Humidity vs AQI	-0.11

Particulates dominate the AQI signal; weather is secondary but still included as model inputs.

6.5 Graph 4: Pollutant / EPA AQI Distribution

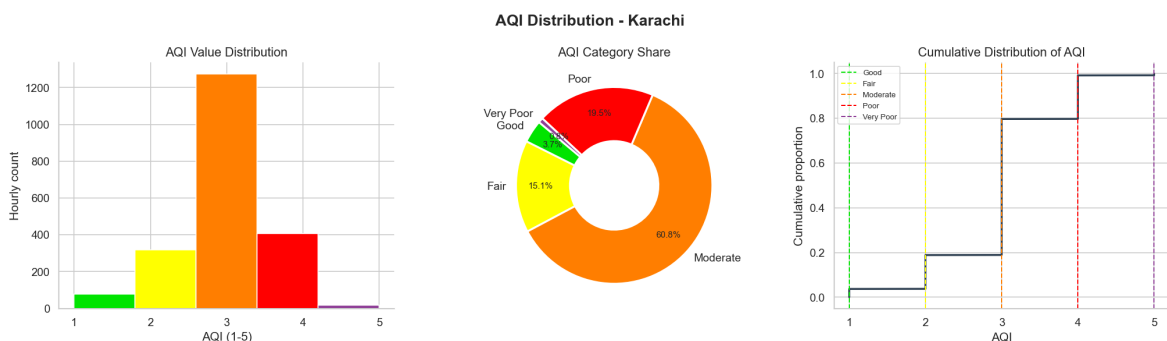


Figure 6: Pollutant / EPA AQI Distribution

Finding: Computed EPA AQI averages ~ 78 with peaks near **177** in mid-March. PM2.5 is the dominant pollutant on $\sim 73\%$ of hours. Daily PM2.5 and PM10 fre-

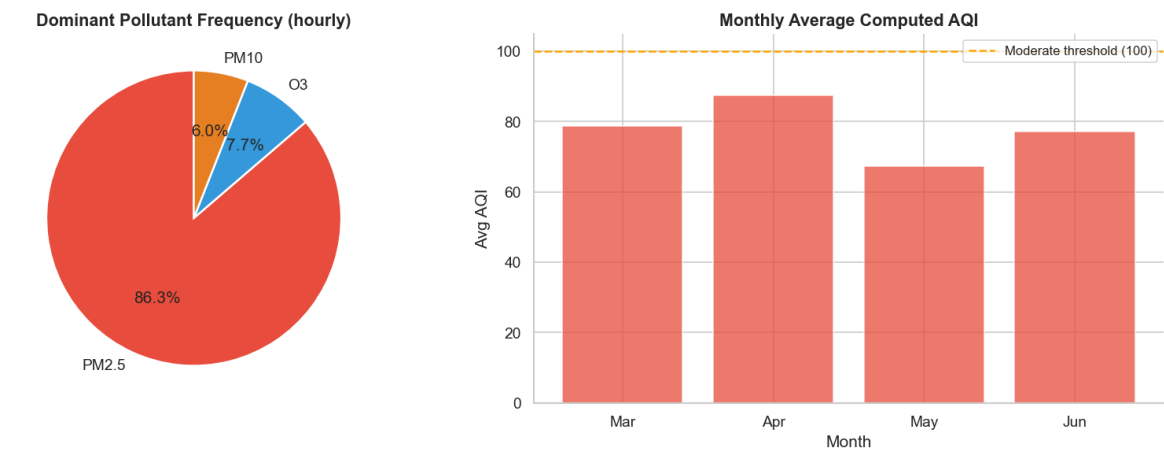


Figure 7: Pollutant / EPA AQI Distribution

quently exceed WHO 24 h guidelines (15 and 45 $\mu\text{g}/\text{m}^3$) during March–April — a genuine public-health concern, not a smooth normal distribution.

6.6 Graph 5: SHAP Feature Importance (Dashboard)

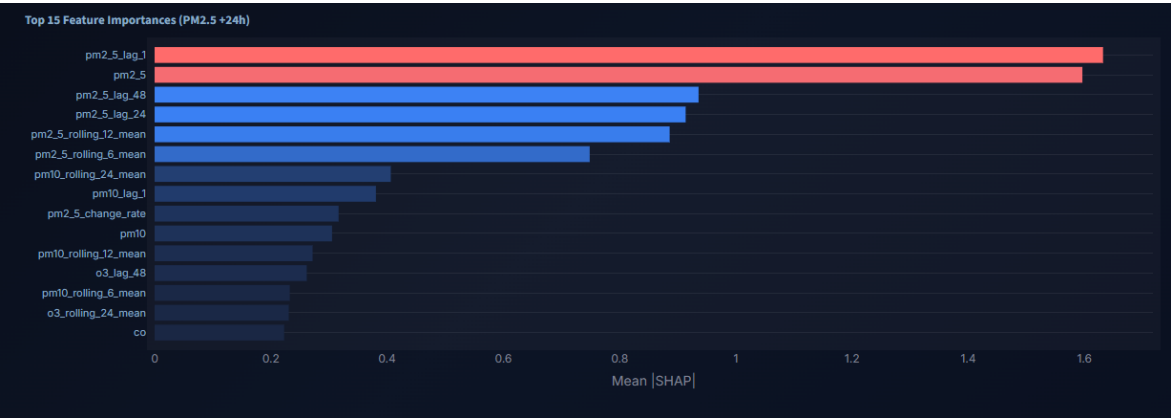


Figure 8: SHAP Feature Importance

Finding (EDA preview + dashboard SHAP): For +24 h PM2.5 forecasting, the strongest drivers are current pm2_5 concentration, pm2_5_lag_24, pm2_5_lag_48, pm2_5_rolling_24_mean, and pm10 and meteorology (humidity, wind_speed) as secondary signals. This aligns with autocorrelation analysis and justifies the lag/rolling feature set.

6.7 EDA Summary of Insights

Table 12: EDA insights and modelling implications

Insight	Implication
PM pollutants dominate AQI	Prioritise PM2.5 / PM10 in modelling and dashboard
Strong temporal persistence	Lag + rolling features essential
Diurnal pattern exists	Hour-of-day feature validated
Episodic spikes, not smooth trends	Compare both regularised linear models and tree ensembles
Weak weekly seasonality	Weekday feature low impact but retained
Time-based split mandatory	Random CV would leak future lag values

7 Model Training

Training is implemented in `src/train.py` and triggered daily by GitHub Actions.

7.1 Forecast Targets

4 pollutants $\times$ 3 horizons = 12 supervised tasks:

Table 13: Forecast targets by pollutant and horizon

Pollutant	Horizons
PM2.5 (<code>pm2_5</code>)	+24 h, +48 h, +72 h
PM10 (<code>pm10</code>)	+24 h, +48 h, +72 h
O ₃ (<code>o3</code>)	+24 h, +48 h, +72 h
NO ₂ (<code>no2</code>)	+24 h, +48 h, +72 h

Target column format: `target_{pollutant}_{24|48|72}h`

7.2 Algorithms Evaluated (per Target)

Table 14: ML algorithms evaluated per target

#	Model	Notes
1	Ridge Regression (+ StandardScaler)	Linear baseline with correlated lags
2	Random Forest Regressor	Shallow trees, subsampling
3	Gradient Boosting Regressor	Regularised boosting
4	XGBoost Regressor	L1/L2 penalties, tuned depth

Total experiments per training run: $12 \times 4 = 48$

Since the dataset contains approximately 2,000 hourly samples, classical machine learning algorithms were selected instead of deep learning models. Tree-based models such

as Random Forest, Gradient Boosting, and XGBoost generally perform well on small-to-medium structured tabular datasets, while deep learning models usually require much larger datasets to generalise effectively.

7.3 Training Protocol

Table 15: Training protocol settings

Setting	Value
Train/test split	Time-ordered 80 / 20 (no shuffle)
Holdout	Last 20% of chronology
Label filter	Per-target <code>.dropna()</code> — maximises rows per horizon
Overfitting rule	Flag when both test/train RMSE ratio > 1.5 and R^2 gap > 0.4
Selection	Prefer non-overfitting models; else lowest test RMSE

Hyperparameters are tuned for $\sim 2k$ hourly rows and ~ 46 features (see `MODEL_CONFIG` in `train.py`).

8 Model Evaluation

Metrics (MAE, RMSE, R^2) are computed on **train and test** sets for every candidate. The winning model per target is saved with full diagnostics in MongoDB `model_registry` and Hugging Face metadata.

For each of the **12 prediction targets**, four algorithms were evaluated: Ridge Regression, Random Forest, Gradient Boosting, and XGBoost. Total: $12 \times 4 = 48$ model experiments.

The **12 best models** (one per target, lowest test RMSE among non-overfitting candidates when available) are stored in the **Hugging Face Model Registry** and referenced from MongoDB.

8.1 Best Models: Summary Table

Table 16: Best model per forecast target - latest training run

Target	Best Model	Test RMSE	Test MAE	Test R^2
target_pm2_5_24h	Random Forest	5.3007	4.2833	-0.4967
target_pm2_5_48h	Random Forest	5.5129	3.9727	-0.1322
target_pm2_5_72h	Gradient Boosting	4.8749	3.6338	+0.3961
target_pm10_24h	Ridge Regression	38.3592	28.9942	-1.7050
target_pm10_48h	Ridge Regression	35.5178	28.2349	-1.0031
target_pm10_72h	Ridge Regression	35.8607	28.7841	-0.6672
target_o3_24h	Ridge Regression	14.2729	12.0910	-2.6419
target_o3_48h	Ridge Regression	13.1736	10.5458	-2.2204
target_o3_72h	Ridge Regression	19.0673	15.4177	-5.7218
target_no2_24h	Gradient Boosting	0.0125	0.0099	+0.6838
target_no2_48h	Random Forest	0.0182	0.0147	+0.3057
target_no2_72h	Random Forest	0.0175	0.0153	+0.3596

Implementation Note

Ridge Regression is persisted in metadata as `LinearRegression` because it is implemented as a `StandardScaler + Ridge` pipeline in `train.py`.

Negative R^2 values on some targets indicate that short-term pollutant spikes are difficult to generalise from only 90 days of data. These models are still evaluated using RMSE/MAE and retained because final AQI calculation depends primarily on dominant pollutants, especially PM2.5 and PM10.

8.2 Evaluation Highlights

- **Time-based split** prevents look-ahead bias from lag and rolling features.
- **Overfitting detection** avoids selecting models that memorise train noise.
- **Horizon effect:** longer horizons (+72 h) typically show higher RMSE — expected for atmospheric forecasting.
- **Particulate models** carry the most weight for final AQI because PM drives the dominant sub-index in Karachi.
- **NO₂ and PM2.5 (+72 h)** show the strongest holdout R^2 ; episodic PM10/O₃ spikes and a short test window make some particulate and ozone targets harder to generalise.

9 Model Registry

Each training run registers the winning model through `src/model_registry.py`:

Model Registration Flow

```

train.py
+-- save_model()
    |-- joblib -> models/ (local cache)
    |-- upload -> Hugging Face Hub
    |     models/<target>/<timestamp>/model.pkl
    |     models/<target>/<timestamp>/metadata.json
+-- insert -> MongoDB model_registry (active=True)

```

9.1 MongoDB Metadata (per Model)

Table 17: MongoDB model_registry document fields

Field	Purpose
target	e.g. target_pm2_5_24h
model_name	Algorithm class name
metrics	train/test MAE, RMSE, R^2 , overfit diagnostics
feature_columns	Ordered schema for inference
hf_repo_id, hf_model_path	Hugging Face artefact location
active	Only one active model per target
trained_at	Timestamp for audit

9.2 Hugging Face Hub

Models are stored in a public model repository so Streamlit Cloud and fresh CI runners can download artefacts without a local `models/` folder. Streamlit and GitHub Actions both consume the same registry.

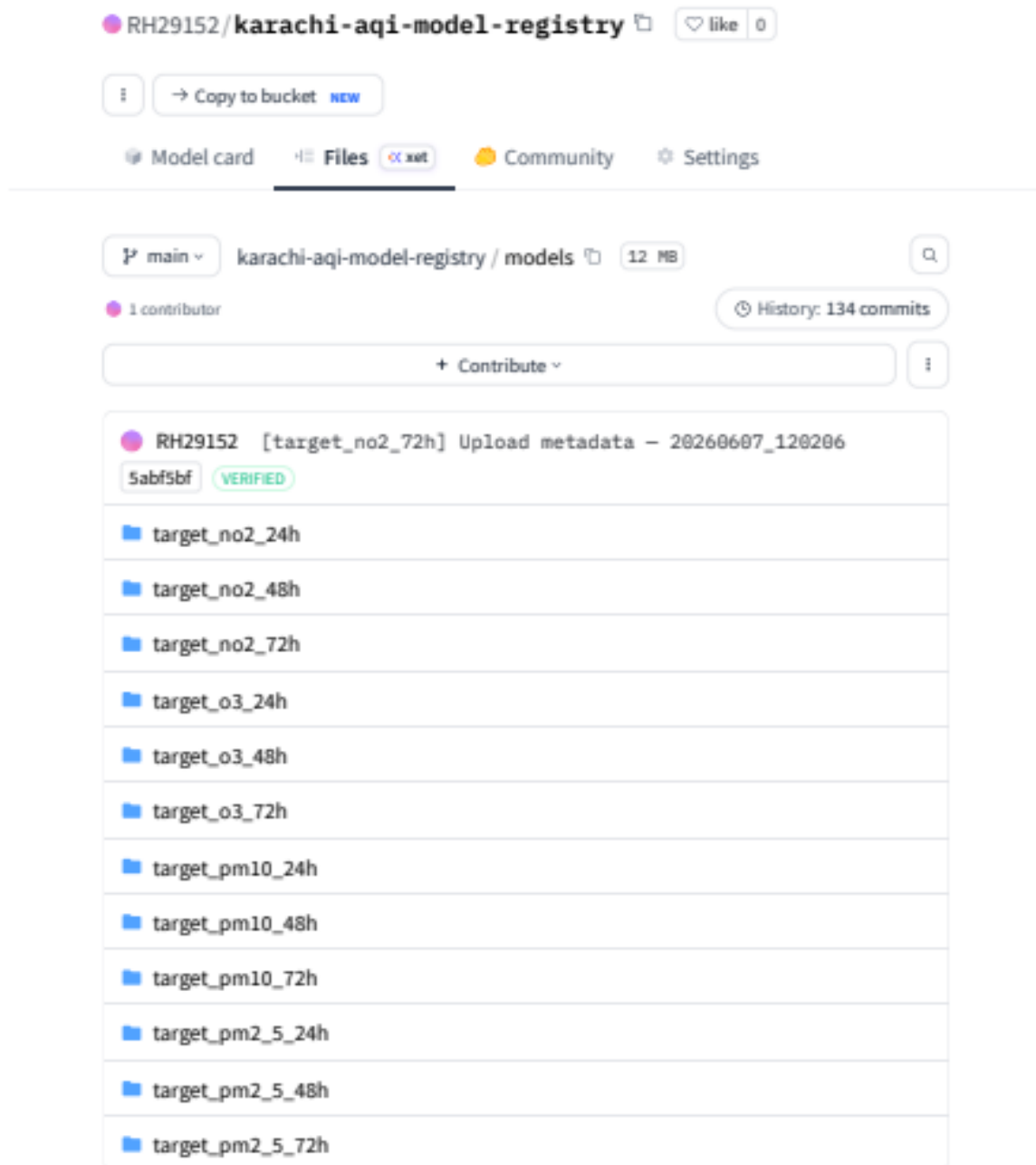


Figure 9: Hugging Face Model Repository

10 Automation / CI-CD

GitHub Actions automate pipeline execution and quality checks. Because GitHub's built-in **schedule** trigger can be delayed by hours on free-tier runners, **cron-job.org** is used as an **external scheduler** to POST to the GitHub **workflow_dispatch** API on time. The workflows themselves still run entirely on GitHub Actions.

Table 18: GitHub Actions workflow summary

Workflow	Schedule	Purpose
Feature Pipeline	Hourly (0 * * * * UTC) via cron-job.org	Catch-up, fetch, incremental features
Training Pipeline	Daily 02:00 UTC (07:00 PKT) via cron-job.org	Retrain 12 targets, HF upload

Both pipeline workflows support **manual dispatch** from the GitHub Actions UI for testing. Secrets (OPENWEATHER_API_KEY, MONGODB_URI, HF_TOKEN, HF_REPO_ID) are injected as environment variables.

10.1 External Scheduler (cron-job.org)

cron-job.org triggers each pipeline by sending an authenticated POST request to:

- Feature: `.../actions/workflows/feature_pipeline.yml/dispatches`
- Training: `.../actions/workflows/training_pipeline.yml/dispatches`

Body: `{"ref":"main"}`. This avoids GitHub schedule queue delays while keeping compute on GitHub Actions runners.

10.2 Feature Pipeline Flow

Feature Pipeline Steps

1. Validate secrets
2. Optional catch-up (≤ 48 missing hours)
 - |-- fetch historical pollution + Open-Meteo weather
 - +-- incremental feature row per missing hour
3. Fetch current OpenWeather snapshot -> raw_data upsert
4. Incremental feature engineering -> features upsert

Catch-up addresses missed runs without requiring a full 90-day rebuild.

10.3 Training Pipeline Flow

Training Pipeline Steps

1. Validate secrets (including HF)
2. `train.py -- 48 candidate fits -> 12 winners`
3. Upload models/ as GitHub artefact (30-day backup)
4. Models + metadata mirrored to Hugging Face + MongoDB

All workflows		Filter workflow runs	
Showing runs from all workflows			
17 workflow runs			
Workflow	Event	Status	Branch
Feature Pipeline (Every 2 Hours)	Feature Pipeline (Every 2 Hours) #6: Manually run by rh29152	Success	main
Feature Pipeline (Every 2 Hours)	Feature Pipeline (Every 2 Hours) #5: Manually run by rh29152	Success	main
Feature Pipeline (Every 2 Hours)	Feature Pipeline (Every 2 Hours) #4: Manually run by rh29152	Success	main
Training Pipeline (Daily)	Training Pipeline (Daily) #3: Manually run by rh29152	Success	main

Figure 10: GitHub Actions Successful Runs

11 Dashboard

URL: <https://aqipredictorbyrashidhussain.streamlit.app/>

The Streamlit app (`app/streamlit_app.py`) reads the latest MongoDB feature row, runs `predict_all_horizons()`, and renders EPA-style AQI for current conditions and three forecast horizons. The dashboard does **not** predict AQI directly — it forecasts pollutant concentrations first, then applies EPA breakpoint formulas.

11.1 Example System Output

Table 19: Example dashboard output at time of report

Output	Value
Current AQI	192
24-hour forecast	AQI 109
48-hour forecast	AQI 94
72-hour forecast	AQI 84
Dominant pollutants	PM2.5 / PM10

The dashboard generates these forecasts by **predicting pollutant concentrations first**, converting each pollutant to an AQI sub-index, and selecting the **highest sub-index** as the final AQI (dominant-pollutant convention).

11.2 Dashboard Sections

Table 20: Dashboard panels and their descriptions

Section	Description
Current AQI	EPA AQI from latest observed pollutants
+24 h / +48 h / +72 h cards	Forecast AQI, category, dominant pollutant
Pollutant breakdown	Per-horizon concentration pills + sub-indices
Forecast timeline	Bar chart Now → +72 h
Historical trends	Computed EPA AQI + 7-day rolling mean
Pollutant trends	Multi-panel concentration time series
Alert banners	Threshold warnings for unhealthy forecasts
SHAP	Top drivers for +24 h PM2.5 model

11.3 Screenshots



Figure 11: Current AQI Hero Card

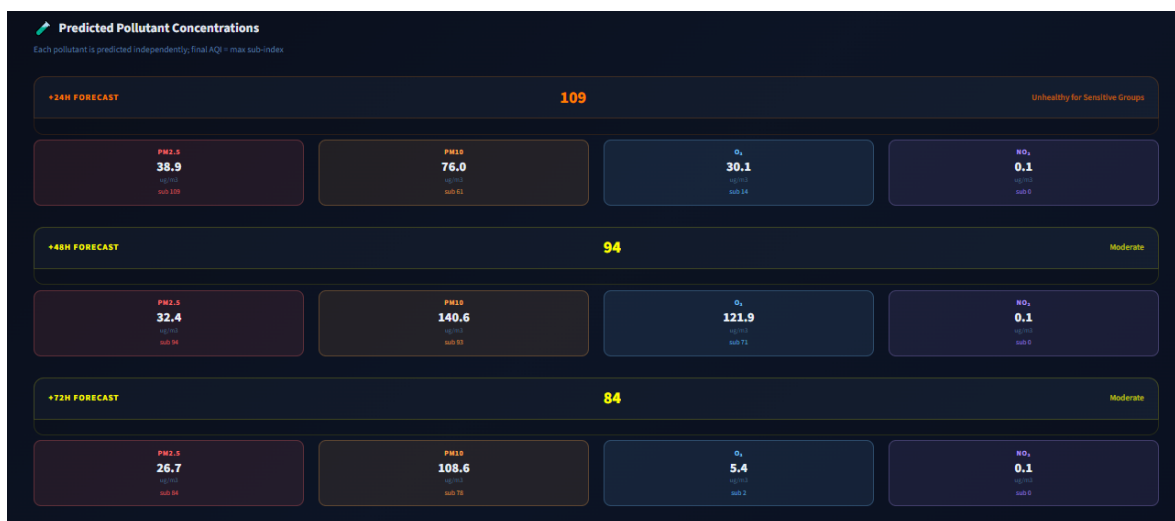


Figure 12: Pollutant Breakdown

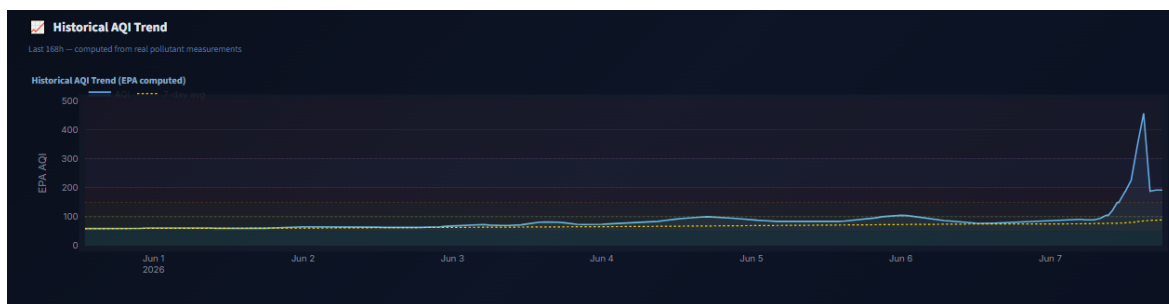


Figure 13: Historical AQI Trend

12 Explainable AI (SHAP)

The dashboard computes **SHAP** values for the active `target_pm2_5_24h` model using the latest ~ 300 feature rows from MongoDB.

12.1 Method

- Tree models $\rightarrow$ `shap.TreeExplainer`
- Linear models $\rightarrow$ `shap.LinearExplainer`
- Mean absolute SHAP $\rightarrow$ ranked horizontal bar chart (top 15 features)

12.2 Interpretation

SHAP complements EDA by showing **which features push the +24 h PM2.5 prediction up or down** on recent data — not just global correlation. Expected top contributors:

Table 21: Expected SHAP feature groups and their influence on PM2.5 +24 h forecast

Feature group	Expected influence
Current PM2.5 / PM10	Strong positive — persistence
PM2.5 lags (1 h, 24 h, 48 h)	Positive during rising episodes
Rolling 24 h means	Smooths recent exposure
Humidity / wind	Dispersion effects
Hour	Diurnal trapping vs. mixing

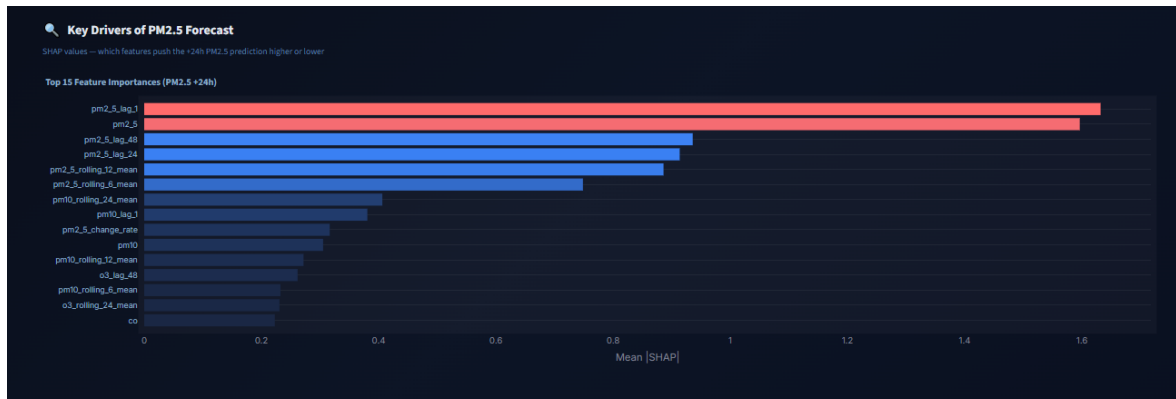


Figure 14: SHAP Bar Chart

13 Challenges and Solutions

Table 22: Challenges encountered and solutions applied

#	Problem	Solution
1	OpenWeather AQI is only a 1–5 category	Switched to pollutant forecasting + AQI calculation via EPA breakpoints
2	OpenWeather category scale hides dominant pollutant	Report max sub-index and name dominant pollutant on dashboard
3	Missed hourly runs / data gaps	Catch-up pipeline fills up to 48 h of gaps automatically
4	Incremental rows lack supervised targets	Batch backfill builds targets; hourly path omits unknown futures
5	Local Windows model paths break on Streamlit Cloud	Resolve cache under <code>models/</code> by filename; download from Hugging Face
6	Streamlit secrets vs <code>os.getenv</code> timing	<code>env_bootstrap.py</code> mirrors <code>st.secrets</code> before config import
7	MongoDB metadata missing HF path (legacy local training)	HF path discovery + re-run Training Pipeline on GitHub Actions
8	GitHub <code>schedule</code> trigger delays on free tier	cron-job.org external scheduler triggers <code>workflow_dispatch</code> on time
9	Data gap (Jun 3–7) during development	<code>backfill.py -rebuild-features</code> + catch-up restored feature parity
10	Plotly / dashboard cache mismatches	Unified <code>load_dashboard_data()</code> cache for features + forecasts

14 Conclusion

This project delivers a **serverless, automated pollutant forecasting + AQI calculation system** for Karachi.

Table 23: Final project deliverables and status

Deliverable	Status
Hourly data ingestion + feature store	MongoDB Atlas (2,091 raw / 2,089 feature rows)
90-day historical bootstrap	<code>backfill.py</code>
12 pollutant forecasting models (4 pollutants $\times$ 3 horizons)	48 experiments $\rightarrow$ 12 winners on Hugging Face
EPA-style 3-day AQI forecasts	+24 h / +48 h / +72 h via sub-index max
Model registry	Hugging Face Hub + MongoDB
CI/CD automation	GitHub Actions (CI, feature, training) + cron-job.org
Public dashboard	Streamlit Cloud (live URL)
Explainability	SHAP on +24 h PM2.5 model
Full EDA notebook	<code>notebooks/01_eda.ipynb</code>

The engineering contribution is the **end-to-end pipeline** — API $\rightarrow$ MongoDB $\rightarrow$ feature engineering $\rightarrow$ pollutant model training $\rightarrow$ AQI calculation $\rightarrow$ registry $\rightarrow$ automation $\rightarrow$ dashboard — not an exhaustive catalogue of EDA plots. EDA informed feature design and model choice; the operational system demonstrates reproducible MLOps practice suitable for extension.

14.1 Future Improvements

- **Multi-city support** (Lahore, Islamabad) with shared feature schema
- **Deep learning** (LSTM / temporal transformers) for sequence-aware forecasting
- **Additional data sources** (e.g. ground-station validation)
- **Alerting** (email/SMS when forecast AQI exceeds thresholds)
- **FastAPI inference endpoint** for programmatic consumers
- **Drift detection** to trigger retraining when error metrics degrade
- **Regulatory averaging windows** for closer alignment with official AQI

A Repository Structure

AQI-Predictor directory layout

```
AQI-Predictor/
|-- .github/workflows/      # CI, feature (hourly), training (daily)
|-- app/streamlit_app.py    # Dashboard
|-- notebooks/01_eda.ipynb # Full EDA
|-- src/                    # Pipeline modules
|-- requirements.txt        # Full local stack
|-- requirements-ci.txt     # GitHub Actions dependencies
+-- FINAL_REPORT.md        # This document
```

B Secrets Reference

Table 24: Environment secrets and their consumers

Secret	Used by
OPENWEATHER_API_KEY	Feature + training + Streamlit
MONGODB_URI	All components
DB_NAME	All components (default: <code>aqi_predictor</code>)
HF_TOKEN	Training upload + Streamlit model download
HF_REPO_ID	Training upload + Streamlit model download

References

1. U.S. Environmental Protection Agency. (2018). *Technical Assistance Document for the Reporting of Daily Air Quality – the Air Quality Index (AQI)*. Office of Air Quality Planning and Standards, Research Triangle Park, NC.
<https://www.airnow.gov/sites/default/files/2020-05/aqi-technical-assistance-document.pdf>
2. U.S. Environmental Protection Agency. (2024). *AQI Breakpoints*.
https://aqs.epa.gov/aqsweb/documents/codetables/aqi_breakpoints.html
3. Zheng, Y., Liu, F., & Hsieh, H. P. (2013). U-Air: When Urban Air Quality Inference Meets Big Data. *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '13)*, 1436–1444.
<https://doi.org/10.1145/2487575.2488188>
4. Bellinger, C., Mohamed Jabbar, M. S., Zaïane, O., & Osornio-Vargas, A. (2017). A systematic review of data mining and machine learning for air pollution epidemiology. *BMC Public Health*, 17, 907.
<https://doi.org/10.1186/s12889-017-4914-3>